Report: Building an Image Classifier using CNN (MNIST Dataset)

🔹 1. Objective

The objective of this project is to build a **Convolutional Neural Network (CNN)** model to classify handwritten digits (0–9) using the MNIST dataset.
Additionally, the model is tested using custom handwritten digit images to evaluate its real-world performance.

🔹 2. Dataset Description

- **Dataset Name:** MNIST Handwritten Digits Dataset
- **Training Samples:** 60,000 images
- **Testing Samples:** 10,000 images
- **Image Size:** 28 × 28 pixels
- **Type:** Grayscale images

Each image represents a digit from **0 to 9**.

🔹 3. Data Preprocessing

The dataset was preprocessed to make it suitable for CNN input:

Steps performed:

- **Normalization:** Pixel values scaled from (0–255) → (0–1)
- **Reshaping:**
  From `(28, 28)` → `(28, 28, 1)` to include channel dimension

One-hot encoding:
Labels converted into categorical format using:
keras.utils.to_categorical(y_train, 10)

Reason:

- Helps model learn efficiently
- Matches CNN input requirements

🔹 4. CNN Model Architecture

The model was built using multiple layers:

Architecture:

1. **Conv2D Layer (32 filters, 3×3)**
   - Extracts basic features like edges
2. **MaxPooling Layer (2×2)**
   - Reduces spatial dimensions
3. **Conv2D Layer (64 filters, 3×3)**
   - Extracts complex patterns
4. **MaxPooling Layer (2×2)**
   - Further reduces size
5. **Flatten Layer**
   - Converts 2D data into 1D vector
6. **Dense Layer (128 neurons)**
   - Learns high-level features
7. **Dropout Layer (0.5)**
   - Prevents overfitting
8. **Output Layer (Dense 10, Softmax)**
   - Produces probabilities for digits (0–9)

🔹 5. Model Training

Compilation:

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Training:

```
model.fit(
    x_train, y_train,
    epochs=5,
    validation_split=0.1
)
```

Explanation:

- **Optimizer (Adam):** Updates weights efficiently
- **Loss Function:** Measures prediction error
- **Epochs:** Model trained multiple times on data

🔹 6. Model Evaluation

The model was evaluated on the test dataset:

```
model.evaluate(x_test, y_test)
```

Result:

- Achieved accuracy close to or above **98%**
- Indicates strong generalization ability

🔹 7. Custom Image Testing

To test real-world performance, custom handwritten images were used.

Challenges faced:

- Different background (white vs black)
- Thin strokes
- Poor centering
- Noise and lighting variations

🔹 8. Image Preprocessing for Custom Input

A preprocessing pipeline was implemented:

Steps:

1. Convert image to grayscale
2. Apply Gaussian blur
3. Apply adaptive threshold
4. Detect largest contour (digit)
5. Crop with padding
6. Resize to 20×20
7. Center in 28×28 image
8. Normalize pixel values

🔹 9. Prediction Process

```
img = final_img.reshape(1,28,28,1)
prediction = model.predict(img)
predicted_digit = np.argmax(prediction)
```

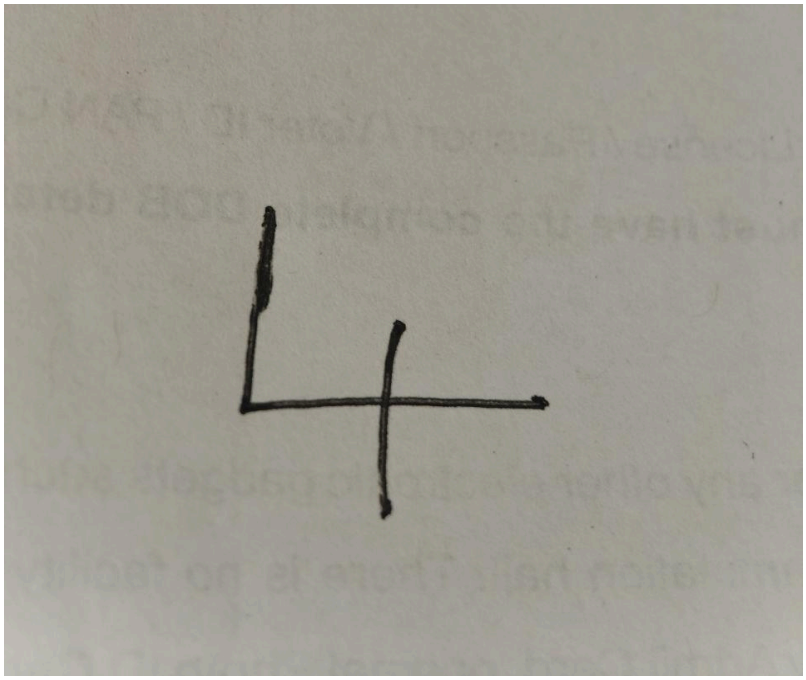
Output:

- Model returns probabilities for each digit
- Highest probability is selected as prediction

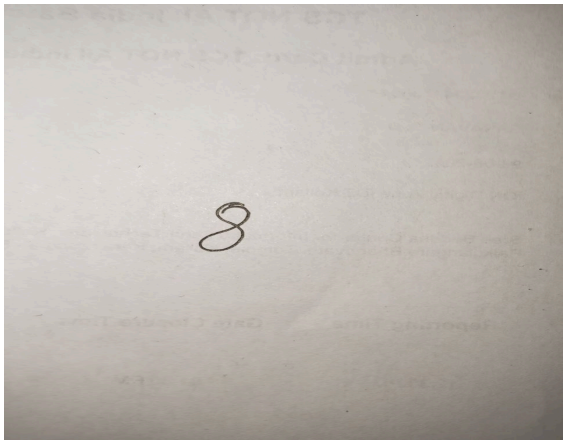
🔹 10. Results on Custom Images

- Initial predictions were inaccurate due to poor preprocessing
- After improving preprocessing:
    - Accuracy improved significantly
    - Predictions became more reliable

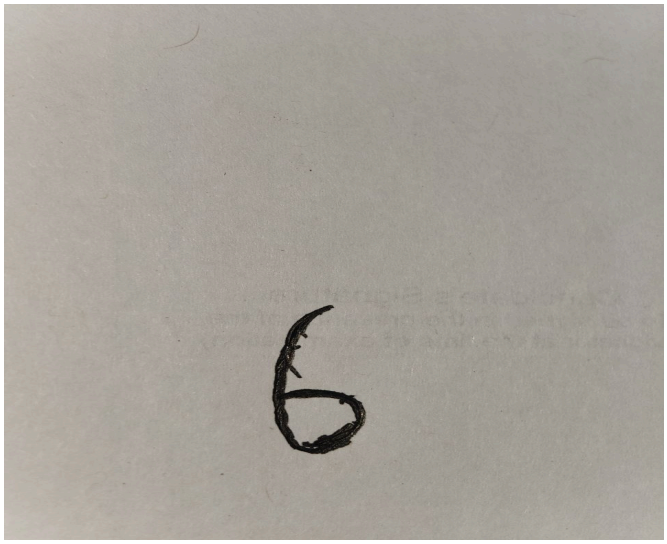
  Still the preprocessing needs a bit more improvement to make the prediction more accurate

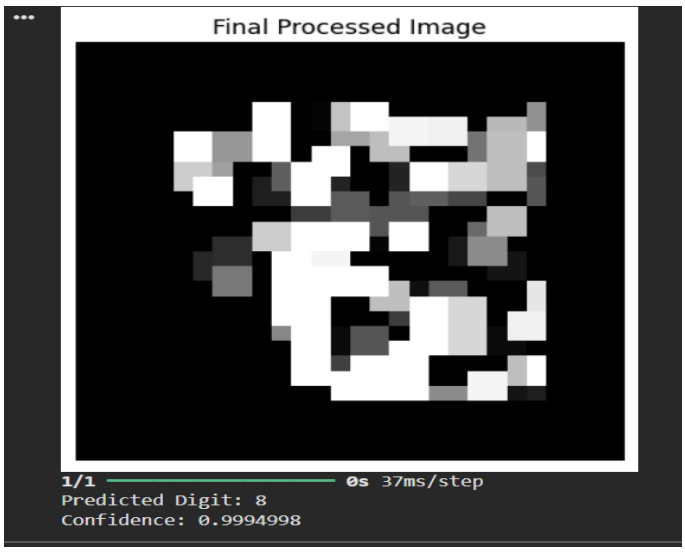


Final Processed Image
1/1 0s 181ms/step
Predicted Digit: 4
Confidence: 0.93157053



Final Processed Image
1/1 0s 37ms/step
Predicted Digit: 8
Confidence: 0.44646484





As the results shows it predicted 4 and 8 correctly but 6 was predicted incorrectly as 8

🔹 11. Key Learnings

- CNNs require **proper input format**
- Preprocessing is **as important as the model**
- Matching input style with training data (MNIST) is critical
- Feature extraction happens layer by layer:
    - Early layers → edges
    - Middle layers → shapes
    - Final layers → classification

🔹 12. Conclusion

A CNN-based image classifier was successfully developed using the MNIST dataset.
The model achieved high accuracy on test data and was further enhanced to handle custom images through improved preprocessing techniques.

The project demonstrates:

- Effective use of CNN for image classification
- Importance of preprocessing in real-world applications
- Strong generalization capability of deep learning models

✅ Final Outcome

- Model Accuracy ≥ 98%
- Successful classification of MNIST digits
- Improved performance on custom images